Lecture 4: Use Case Diagrams

SE2030 – Software Engineering

Lesson Learning Outcomes

- Introduction to **requirement specification – Graphical Notations**
- Identify and describe **key components** of use case diagrams
- Applying **use case diagrams in real world applications** and Construct use case diagrams for real-world software applications.
- Develop detailed **use case scenarios**.

Requirements Specification

- Document Requirements identified in requirements gathering and analysis phase.

Using Graphical Notations

- Use Case Diagrams and Use Case Scenarios
- Activity Diagrams

What is a Use Case Diagram?

- Use Case Model;
  - Graphically represent the proposed functionality of the new system.
  - Use Case Model captures the functional requirements of a system.
  - Help to demonstrate the high-level behavior of the proposed system to the clients

Use Cases for Requirements Engineering

- Use case modelling support Requirements Specification

- Use cases act as a means of communicating with stakeholders about what the system is intended to do.
  - It is an excellent way to communicate to management, customers, and other non-development people:
  - WHAT a system will do when it is completed.
  - But....it does not go into detail of HOW a system will do anything.

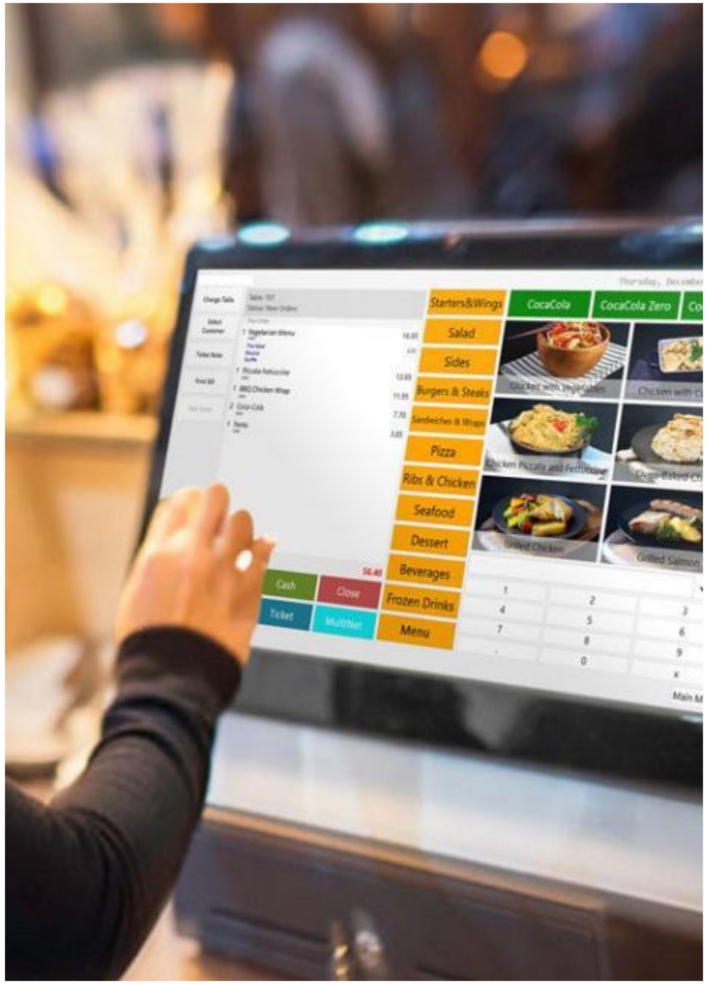
Components of a Use Case Diagram

- To construct a Use Case diagram, **there are FOUR basic components.**
  - **System**: something that performs function(s).
  - **Actors**: the roles adopted by those participating.
  - **Use Cases**: high level activities to be supported by the system.
  - **Relationships / Links**: which actors are involved in which use cases (dependency, generalization, and association).

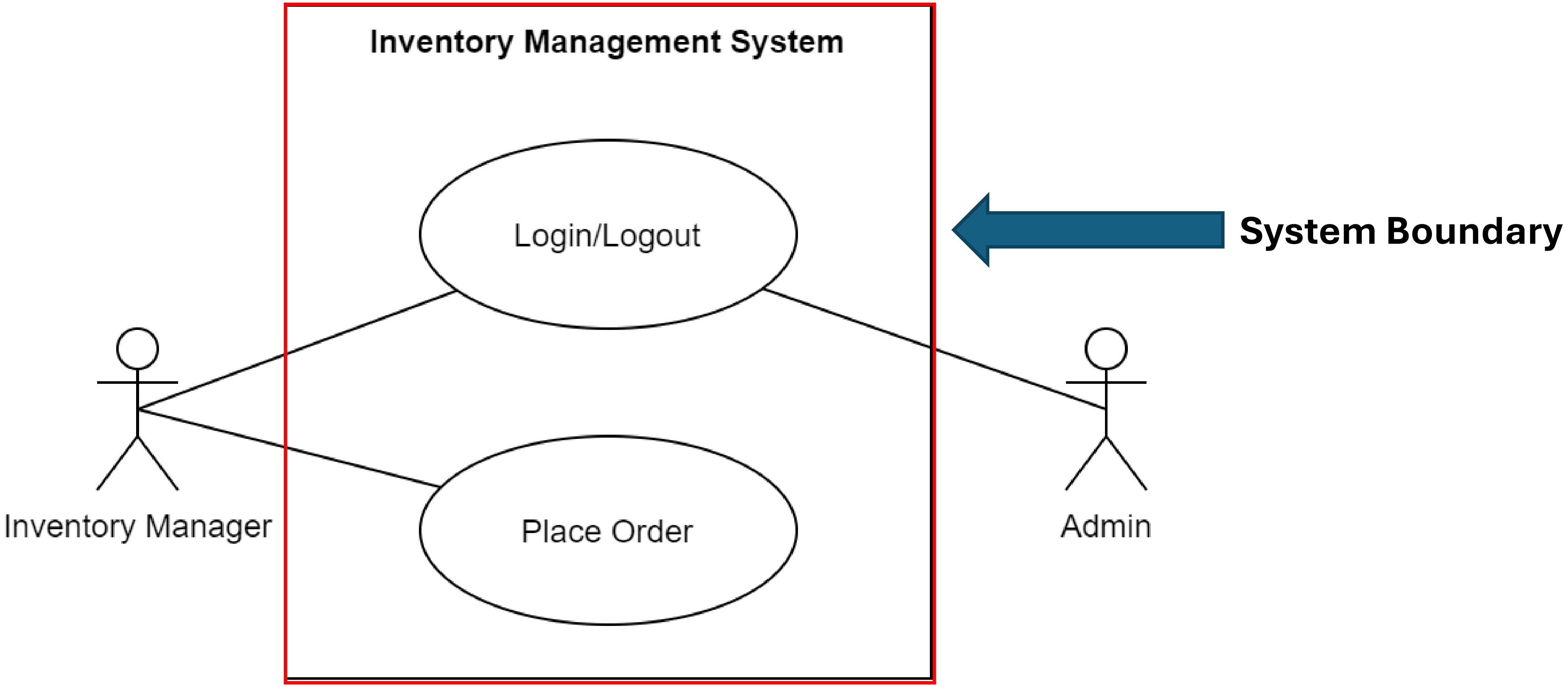
1) System

- **System** is something which perform function(s).

- **System Boundary** Represents the boundary between the (physical) system and the actors who interact with the (physical) system.

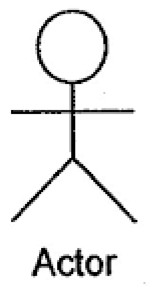


System - Example



2) Actors

- A Use Case Diagram shows the interaction between the system and entities external to the system. These external entities are referred to as Actors.
- Actors represent roles which may include human users, external hardware or other systems.
- Actors have direct interactions with the system
- Notation →



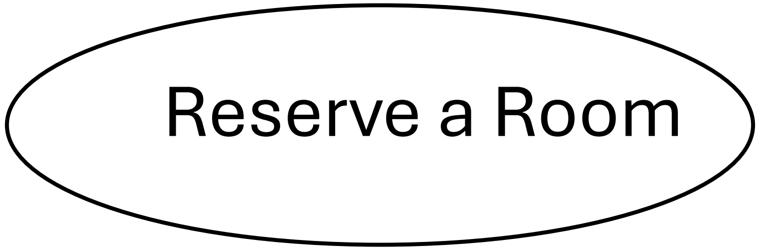
Activity 01

- Identify Actors for a Hotel Management System.

3) Use Case

- A Use Case is a unit of behavior in the proposed system
- It represents a unit of interaction between a user and the proposed system.
- Use case name typically has a verb-noun phrase

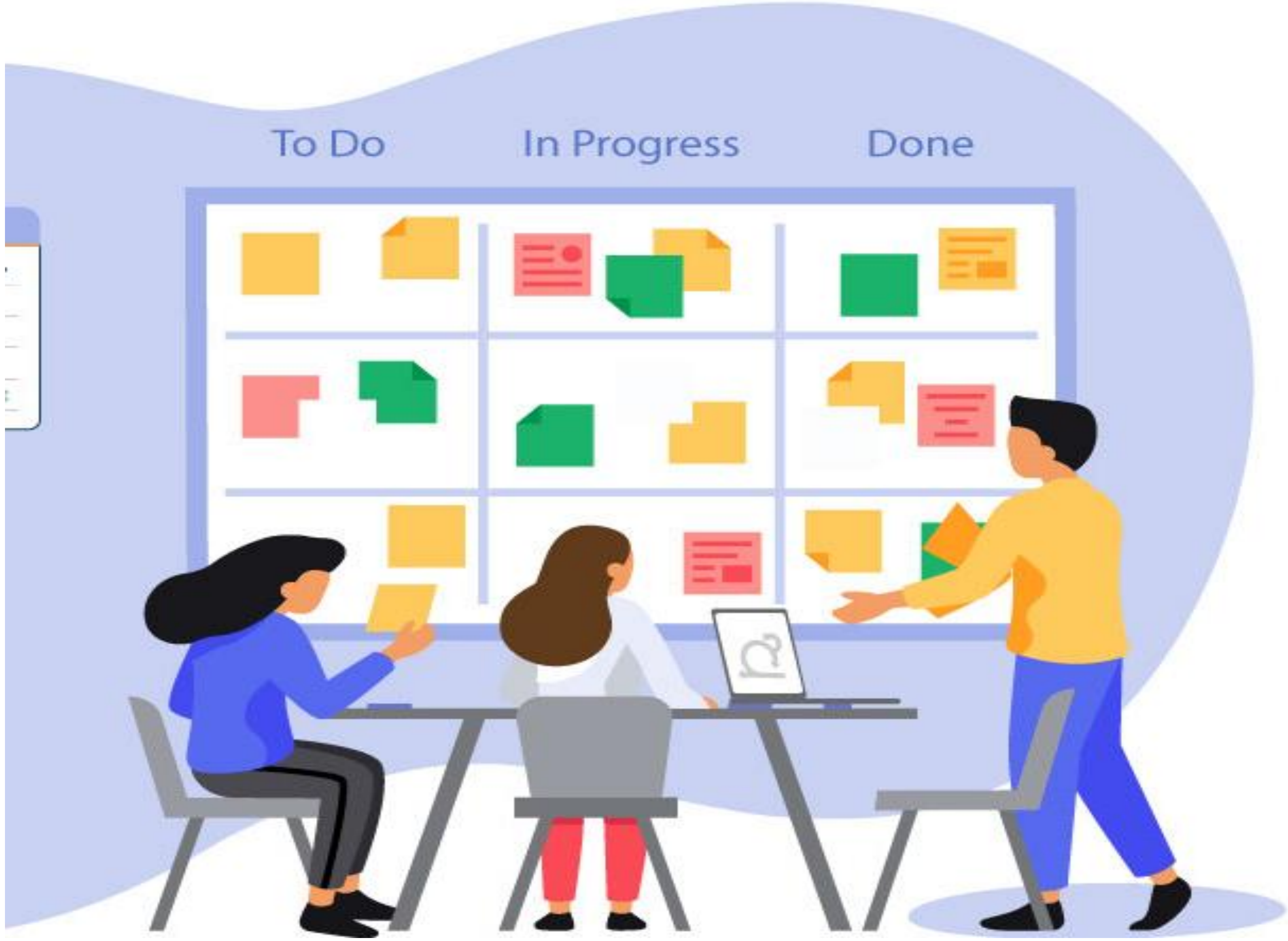
- Notation →



How to Identify a Use case?

- Consider what each actor requires of the system.
- For each actor, human or not, ask yourself the following questions in order to figure out the relevant use cases.
    - What are the primary tasks the actor wants the system to perform?
    - Will the actor create, store, change, remove, or read data in the system?
    - Will the actor need to inform the system about sudden, external changes?
    - Does the actor need to be informed about certain occurrences in the system?
    - Will the actor perform a system start-up or shutdown?

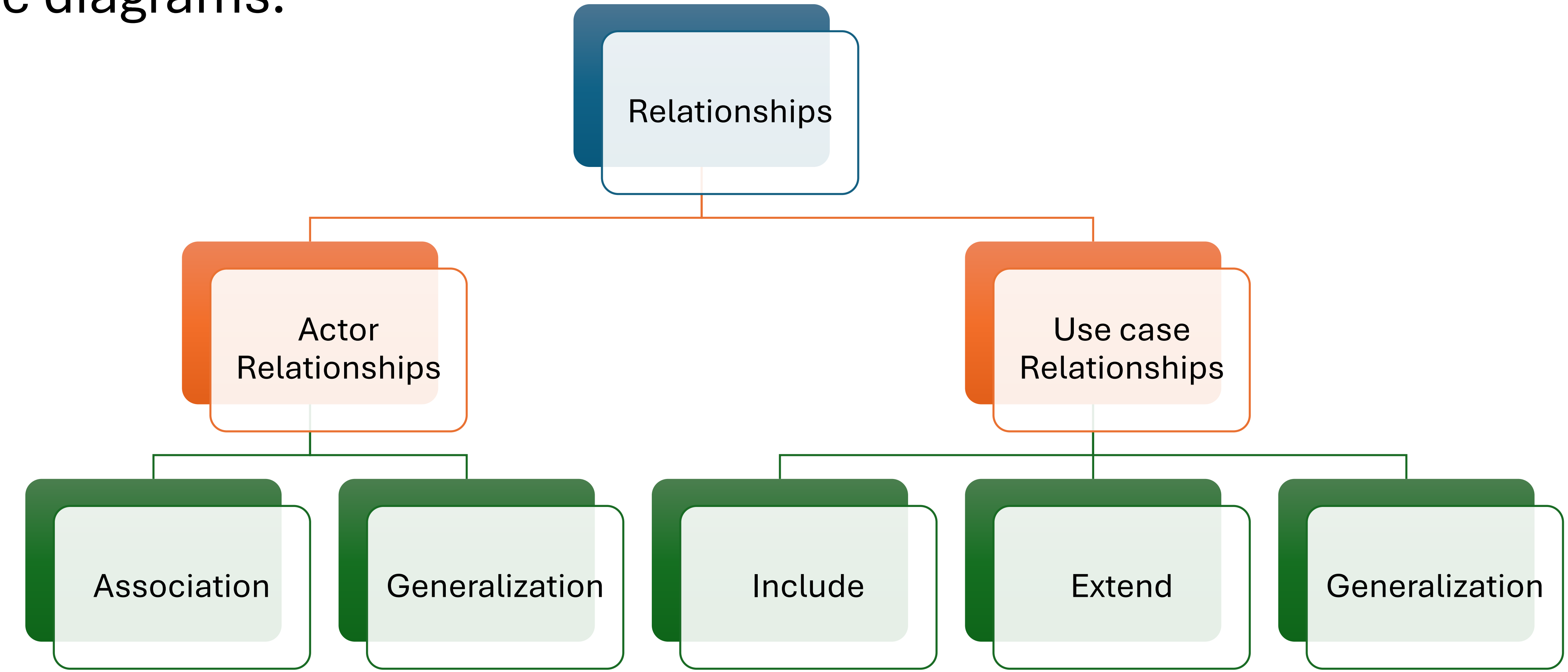
Activity 02



- Identify Use Cases for a Hotel Management system.

4) Relationships

- Below mentioned are the main types of relationships used in use case diagrams.

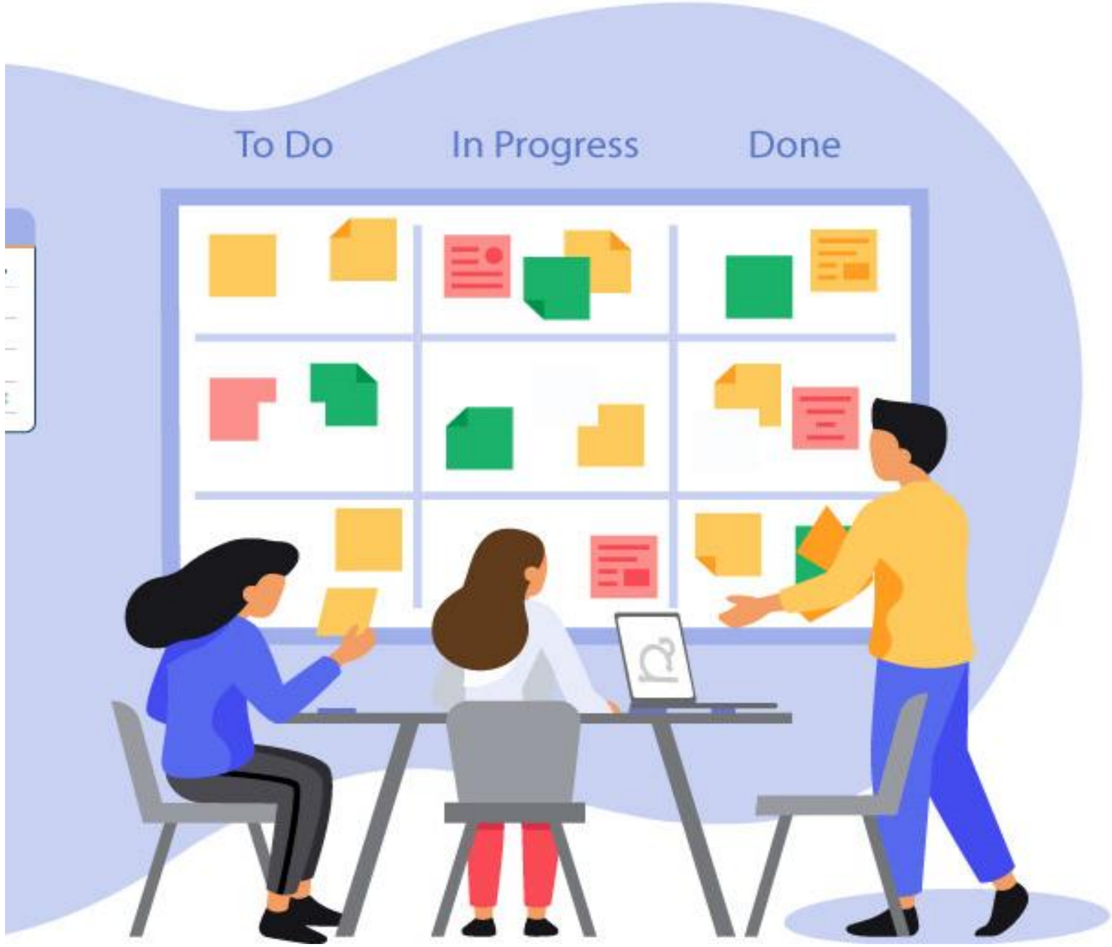


Actor to Use case Relationships

Association.

- indicates that an actor participates in (i.e. communicates with) the use case.

Activity 03



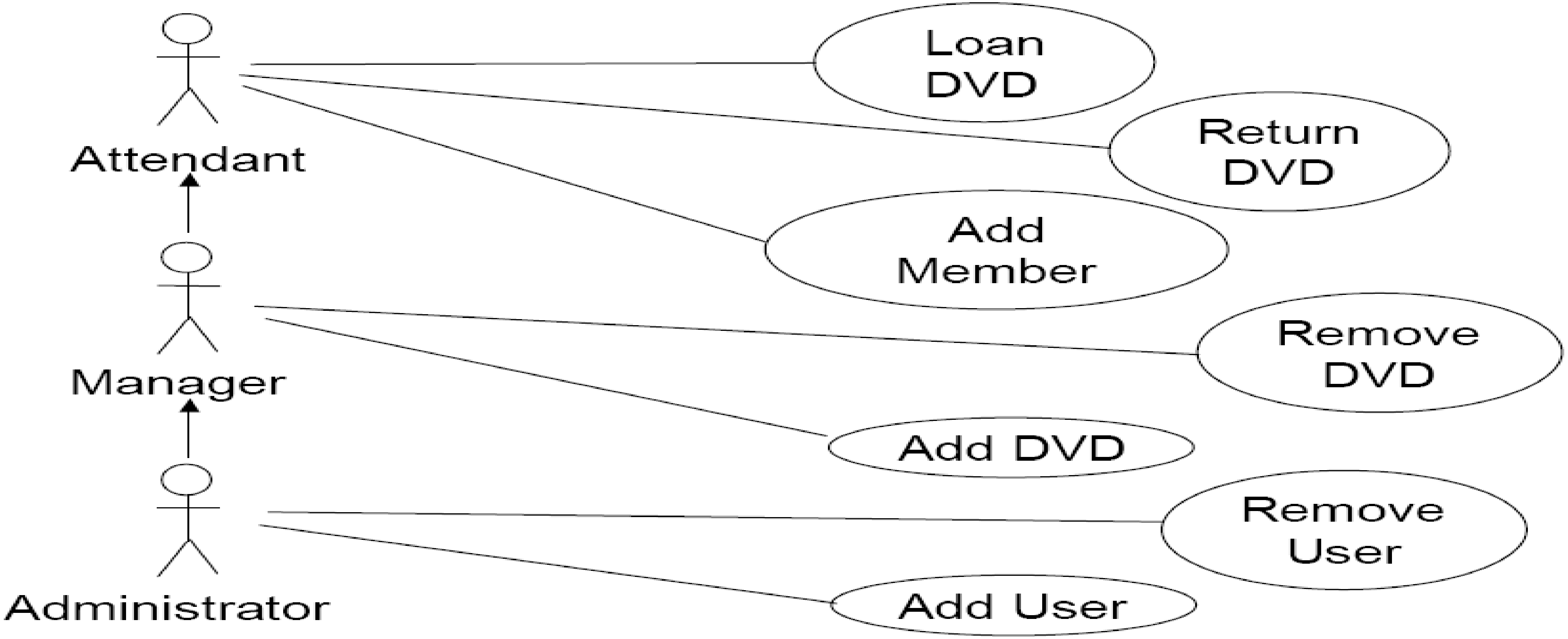
- Draw the Actors and Associations for the Hotel Management system

Actor to Actor Relationships

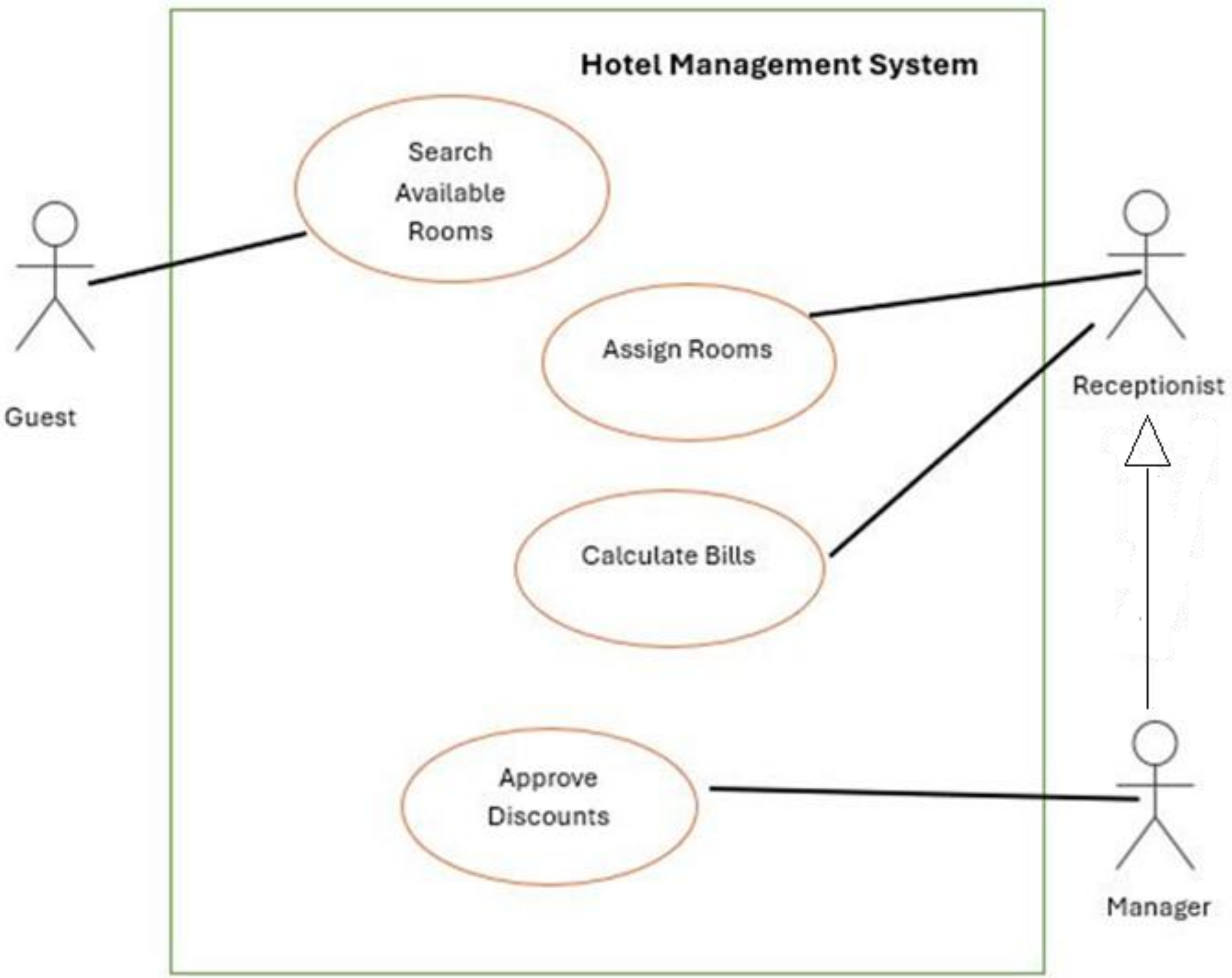
Generalization.

- Actor Generalization is drawn from the concept of inheritance in Object Oriented Programming.
- A child actor Inherits all of the characteristics and behavior of the parent actor.
- Can add , modify, or ignore any of the characteristics and behaviors of the parent actor.

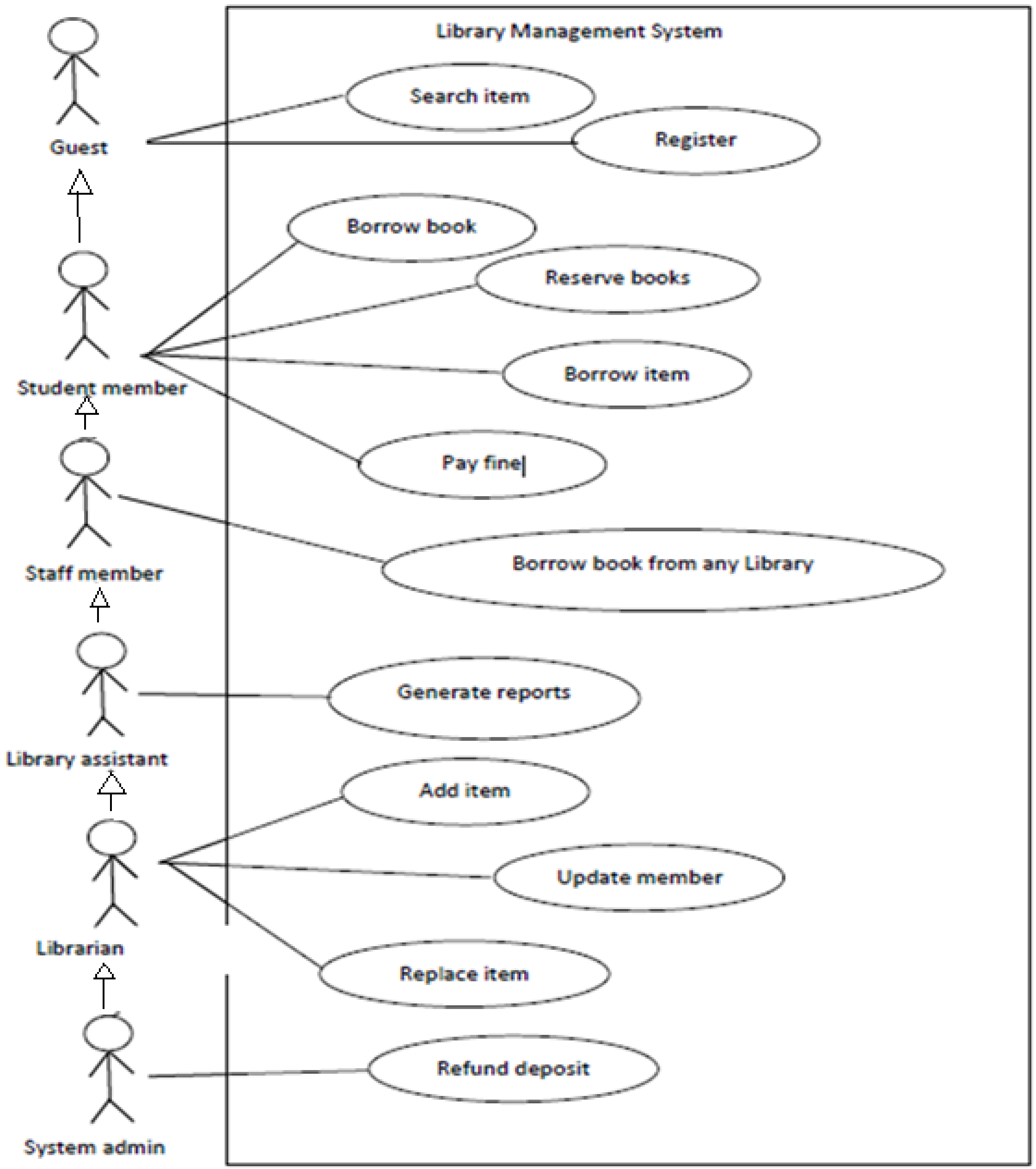
Who has the most rights in the system?



Example – Hotel Management System



More Examples on Actor Generalization

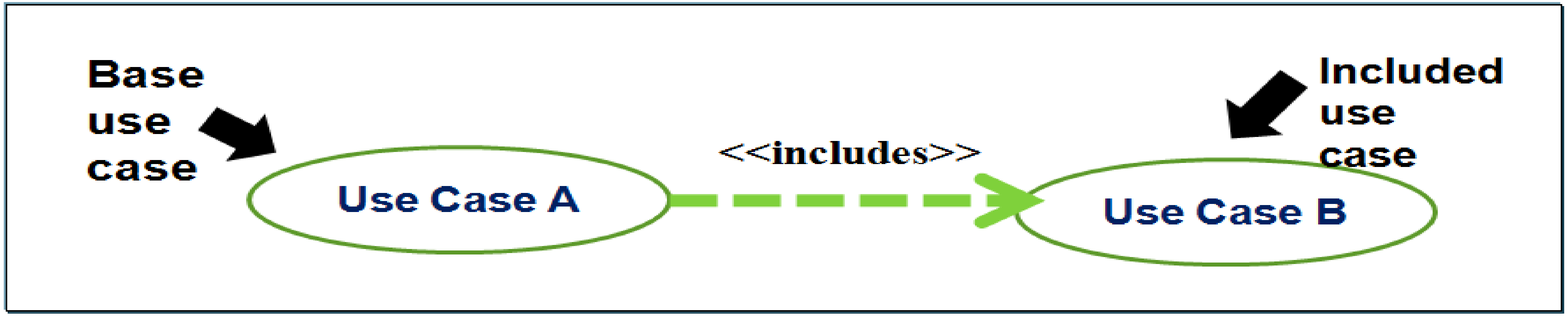


Include Relationship

1) Include

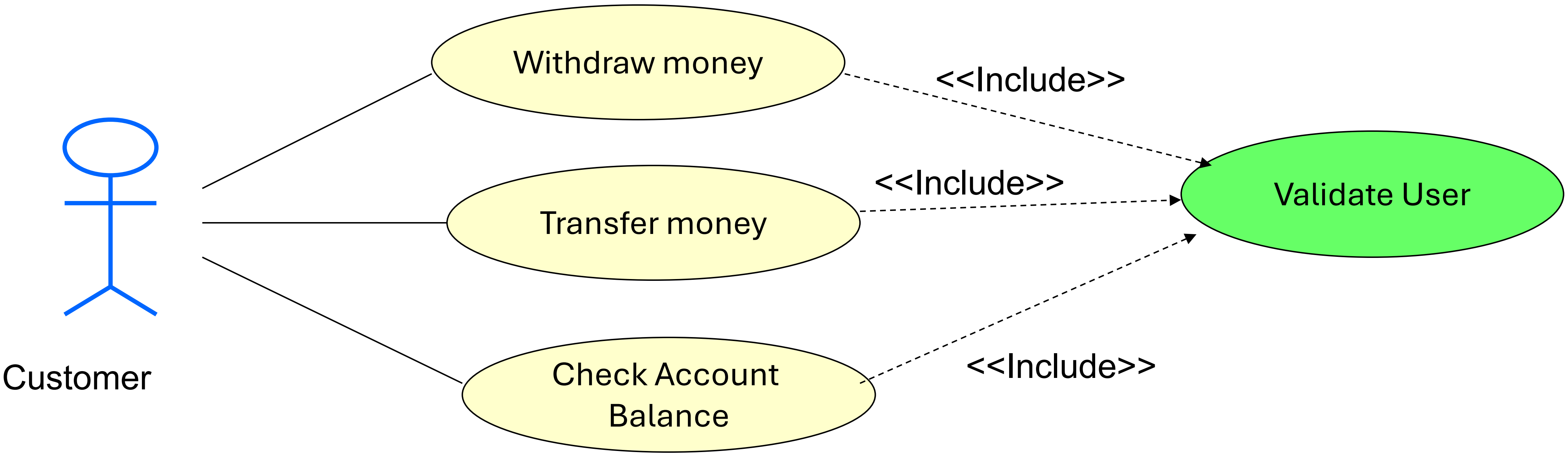
- The base use case explicitly incorporates the behavior of another use case at a location specified in the base.

- The included use case never stands alone. It only occurs as a part of some larger base that includes it.



Include Relationship

- Enables us to avoid describing the same flow of events several times by putting the common behavior in a use case of its own.

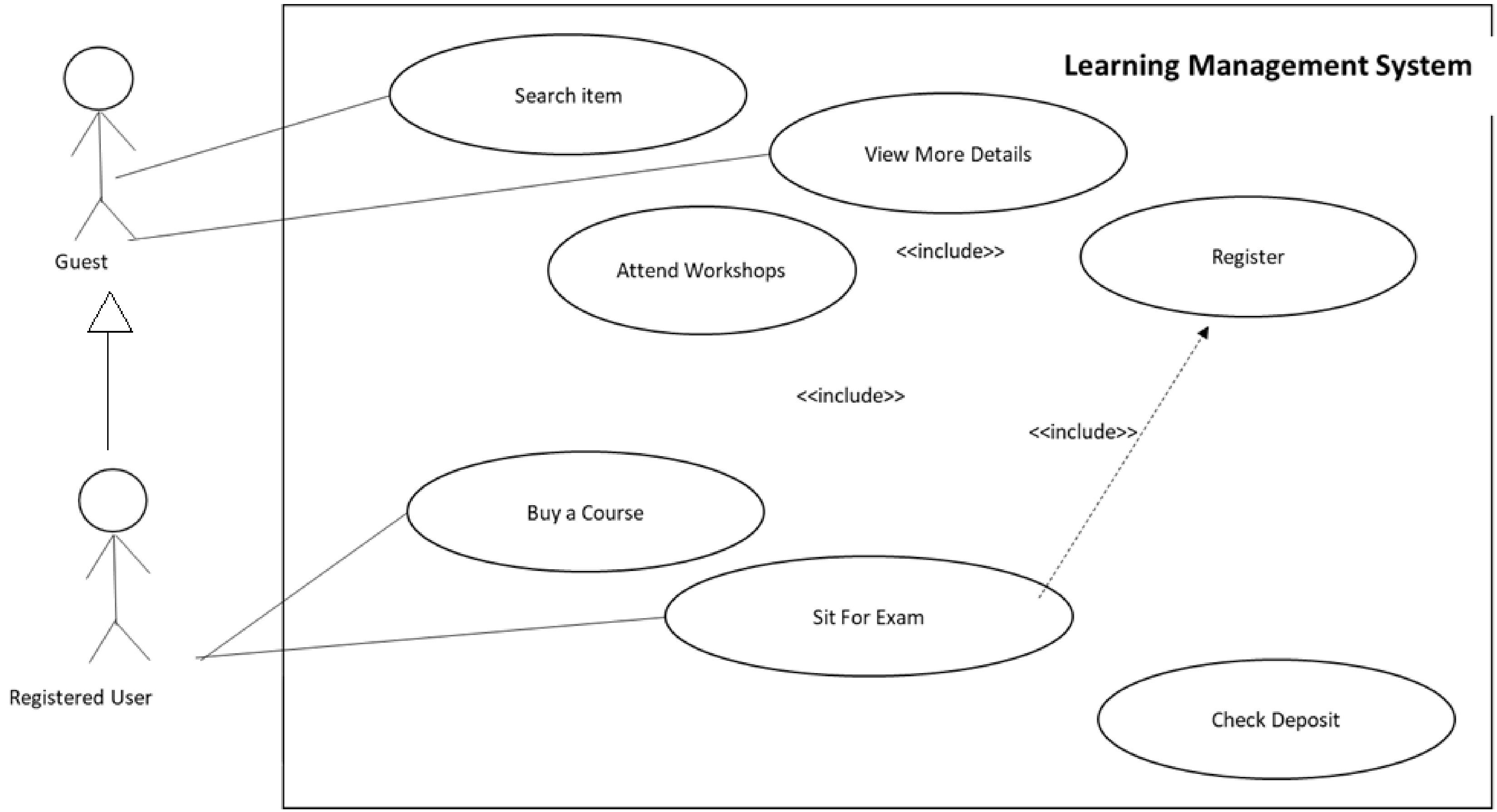


Activity 04

- Learning Management System

A guest can **search** for available courses and **view more details**. If the guest wants to **attend a workshop**, they must **register** in the system to proceed.

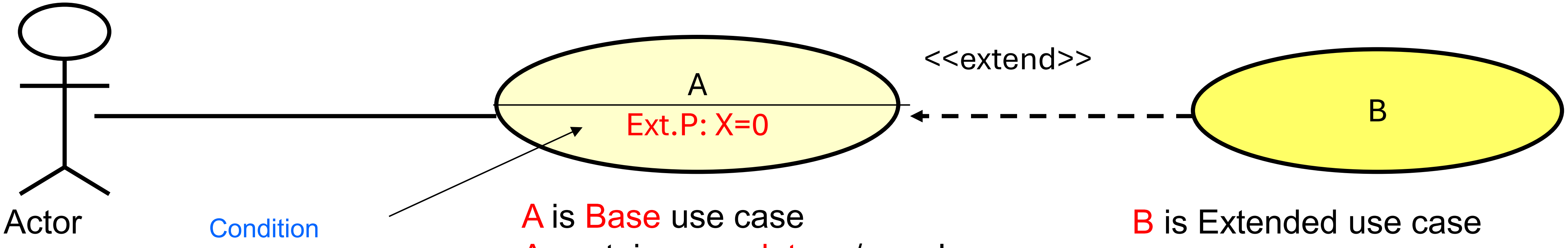
Include Relationships - More Examples



Extend Relationship

2) Extend

- The base use case implicitly incorporates the behavior of another use case at certain points called extension points.
- The base use case may stand alone, but under certain conditions its behavior may be extended by the behavior of another use case.

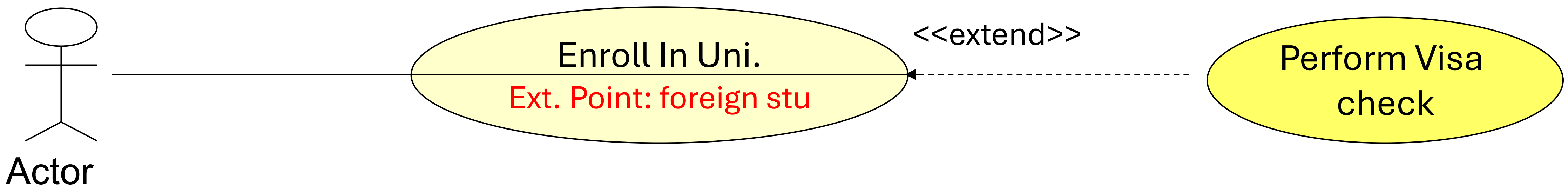


A is Base use case
A contains mandatory / regular behavior

B is Extended use case
B is optional behavior

Extend Relationship

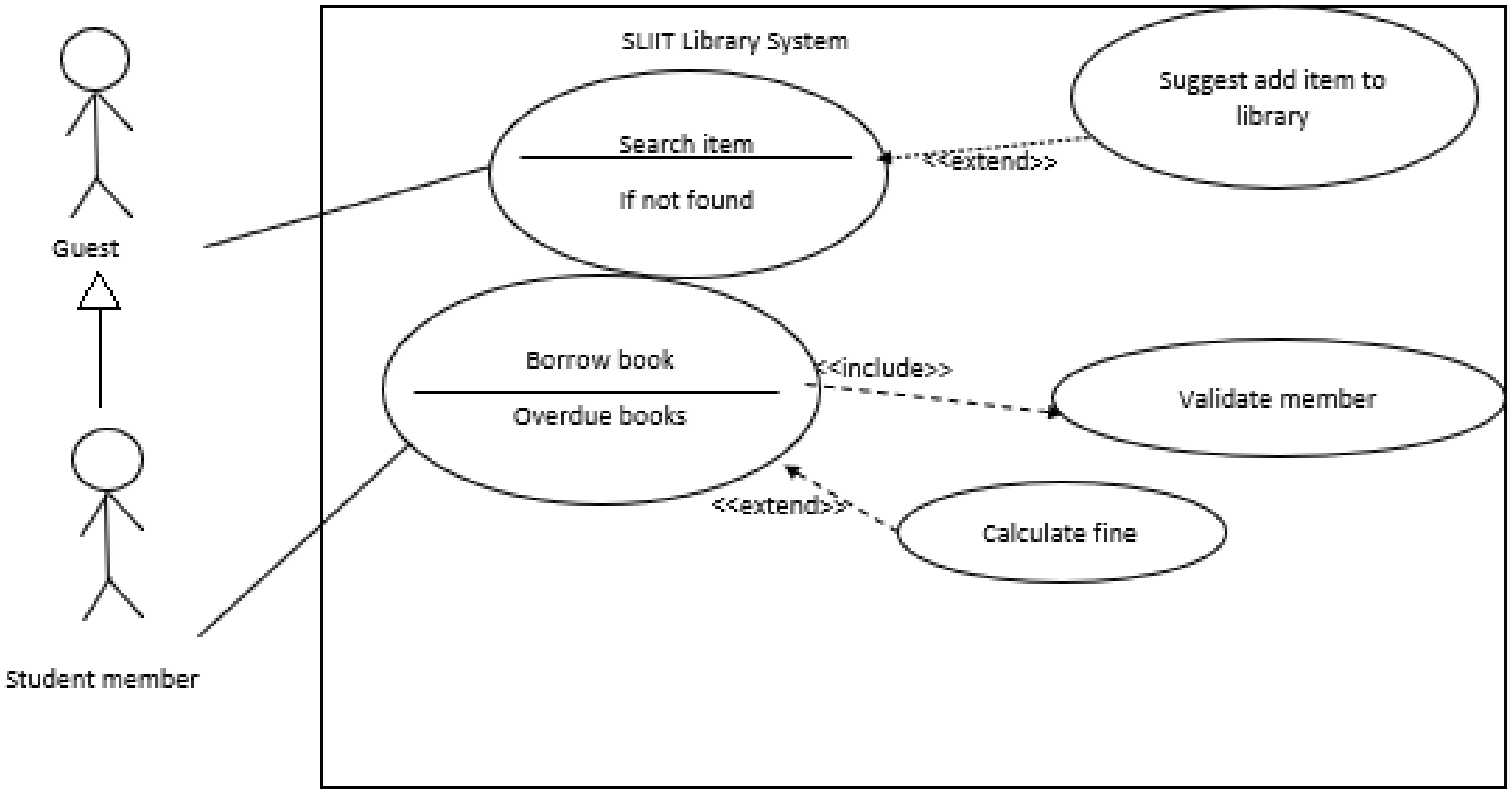
- Eg:- When a student get enrolls in the university, they perform a visa check if he/she is a foreign student.



Activity 05

- “Research Rabbit” is a popular system that helps undergraduate students find supervisors whose expertise matches their research interests. Any user can search for available research groups. While searching, a user may request more information about a research group if interested. If the user’s preferred research area is unavailable, they may suggest a new research area to the system.
- A user can also submit an Expression of Interest (EOI) to proceed with finding a suitable research area and supervisor. Whenever a user submits an EOI, the system provides a prospective student registration link as part of the submission process. If the EOI is not properly completed, the system may also reject the application.

More Examples- Extend Relationships



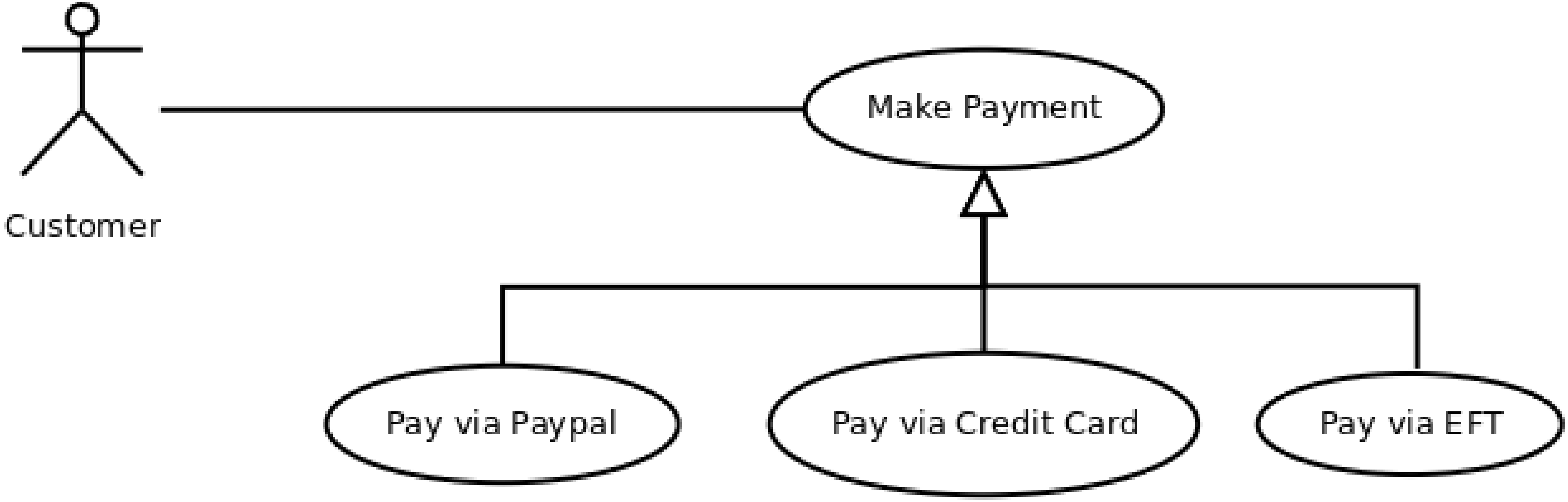
Generalization Relationship

3) Generalization

- The child use case inherits the behavior and meaning of the parent use case.

- The child may add to or override the behavior of its parent.

Generalization Relationship



Activity 06- Student Enrollment System

"St John York University" is a well-known UK-based university that supports students seeking postgraduate study opportunities in the UK. Any user can search for available postgraduate programmes by providing their age, current qualification, area of study, and preferred study level. Users can search for either research programmes or taught programmes. If required, they can also view the enrolment requirements, including language requirements.

A registered student can access all the features available to a normal user. In addition, a registered student can book a discussion in one of three areas: IT, Business, or Science. The university will assign a suitable counsellor according to the selected area, while the counsellor's activities are not handled through this system.

Relationship Summary

Table 6-1: *Kinds of Use Case Relationships*

Relationship	*Function*	*Notation*
association	The communication path between an actor and a use case that it participates in	————
extend	The insertion of additional behavior into a base use case that does not know about it	«extend» - - - - ->
include	The insertion of additional behavior into a base use case that explicitly describes the insertion	«include» - - - - ->
use case generalization	A relationship between a general use case and a more specific use case that inherits and adds features to it	————▷

Activity 07 -Self Study

Draw a partial use case diagram for the given case study.

“Dirgayu” is an application for patients' management. Registered nurse can admit patient through the system. When admitting the patient , system needs vaccination details to complete the process. Also, if foreign patient , system needs a valid passport to proceed with admitting procedure. Once completed the admitting the process system will show causality ward number and send e-card to the ward.

Medical officer who assigned to that ward can check e-card to proceed with the medical appointment. Also, he can enter details to the system according to severity level. So medical record can be categorized to severe redline and low risk blue line. Furthermore, Medical officer can do the all the tasks which registered nurse can do.

Use Case Scenarios

- A Scenario is a formal description of the flow of events that occur during the execution of a Use Case instance. It defines the specific sequence of events between the system and the external Actors.

- There is usually a Main scenario, which describes what happens when everything goes to plan. It is written under the assumption that everything is okay, no errors or problems occur, and it leads directly to the desired outcome of the use-case.

Use Case Scenarios

- Other scenarios describe what happens when variations to the Main scenario arise, often leading to different outcomes.

- So the flow of events should include:
  - How and when the use case starts and ends
  - When the use case interacts with the actors
  - What objects are exchanged
  - The basic flow and
  - Alternative flows (exceptional) of the behavior.

Use Case Sample Template

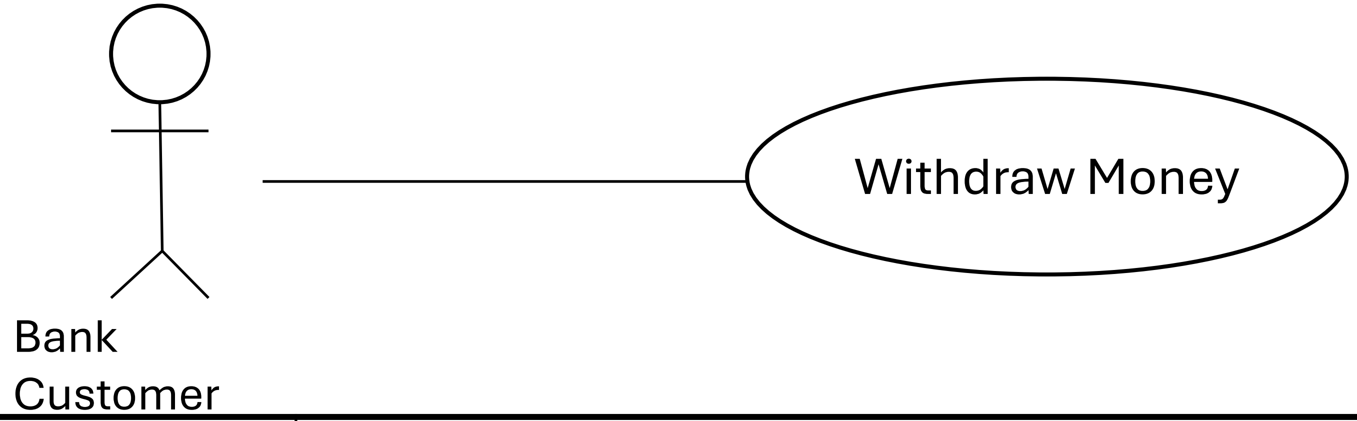
1. Use Case ID and name
2. Characteristic Information
   - Goal in Context
   - Scope
   - Level
3. Pre-Conditions
4. Primary Actor
5. Main Success Scenario Steps
6. Extensions
7. Optional Information

Use Case Specification Template*

Number	*Unique use case number*	
Name	*Brief noun-verb phrase*	
Summary	*Brief summary of use case major actions*	
Priority	*1-5 (1 = lowest priority, 5 = highest priority)*	
Preconditions	*What needs to be true before use case "executes"*	
Postconditions	*What will be true after the use case successfully "executes"*	
Primary Actor(s)	*Primary actor name(s)*	
Secondary Actor(s)	*Secondary actor name(s)*	
Trigger	*The action that causes this use case to begin*	
Main Scenario	**Step**	**Action**
	Step #	*This is the "main success scenario" or "happy path."*
	…	*Description of steps in successful use case* "execution"
	…	*This should be in a "system-user-system, etc." format.*
Extensions	**Step**	**Branching Action**
	Step #	*Alternative paths that the use case may take*
Open Issues	*Issue #*	*Issues regarding the use case that need resolution*

*Adapted from A. Cockburn, "Basic Use Case Template"

Use Case Specification Template Example



Number	1
Name	Withdraw Money
Summary	User withdraws money from one of his/her accounts
Priority	5
Preconditions	User has logged into ATM
Postconditions	User has withdrawn money and received a receipt
Primary Actor(s)	Bank Customer

Continued ...

Trigger	User has chosen to withdraw money	
Main Scenario	**Step**	**Action**
	1	System displays account types
	2	User chooses account type
	3	System asks for amount to withdraw
	4	User enters amount
	5	System debits user’s account and dispenses money
	6	User removes money
	7	System prints and dispenses receipt
	8	User removes receipt
	9	System displays closing message and dispenses user’s ATM card
	11	User removes card
	12	System displays welcome message
Extensions	**Step**	**Branching Action**
	5a OR 5.1	System notifies user that account funds are insufficient
	5b OR 5.2	System gives current account balance
	5c OR 5.3	System exits option
Open Issues	1	Should the system ask if the user wants to see the balance?

Activity 08

- Write a Use Case Scenario for "**Borrowing a Book**"

You could consider the process given below as the manual system procedure.

The member identifies him or herself to the librarian and indicates which books they wish to borrow.

If it is acceptable for them to borrow these books, i.e. they are not marked "for reference only", or the number of books on loan to the customer is less than some predetermined maximum, then the books are loaned to the customer for a specified loan period.

The members loan record is updated to reflect the loaned books. The libraries card index system is updated to show who has borrowed the books.

References

- *Writing Effective Use Cases*
  - *By Dr. Alistair Cockburn*
- UML 2 Bible

Thank You!